

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по лабораторной работе

«Генераторы статических сайтов на Python: публикация результатов экспериментов»

Исследовательское задание T5 — публикуемость и цитируемость результата

Практическое задание P2 — стресс-тест научного контента

Выполнил: студент группы P4209
Павлов Александр Сергеевич

Преподаватель:
Жуков Николай Николаевич

Санкт-Петербург, 2026

Содержание

1. Цель и результаты	3
2. Ход работы	3
3. Пайплайны	4
4. T5. Публикуемость и цитируемость	5
5. P2. Стресс-тест научного контента.....	5
6. Отладка	6
7. Вывод.....	7

1. Цель и результаты

Цель — опубликовать результаты вычислительного эксперимента статическим сайтом на Python-генераторах, автоматизировать сборку и развёртывание на GitHub Pages и отечественных площадках, выполнить исследовательское задание Т5 (публикуемость и цитируемость) и практическое задание Р2 (стресс-тест научного контента на двух генераторах). Полная версия отчёта со скриншотами опубликована на сайтах (раздел «Отчёт»).

Ресурс	Адрес
Репозиторий	https://github.com/ExcaliBBur/static_site
GitHub Pages: MkDocs / Sphinx	https://excalibbur.github.io/static_site/ , .../static_site/sphinx/
Helios ИТМО	https://se.ifmo.ru/~s335989/p2/
Собственный VPS	https://excalibbur.ru/static_site/
Запуски CI (успешные и проваленные)	https://github.com/ExcaliBBur/static_site/actions

2. Ход работы

№	Шаг	Что сделано
1–3	Python, pip, virtualenv	Python 3.13.3, pip 25.0.1, virtualenv 21.13.0 (доустановлен); окружение .venv
4	Зависимости	requirements.txt с точными версиями (15 пакетов); .gitignore (.venv/, site/, _build/, public/, кэш); .gitattributes (LF)
5	Генераторы	MkDocs 1.6.1 + Material 9.7.7; Sphinx 9.1.0 + MyST-Parser 5.1.0 + Furo
6	Локальная сборка	mkdocs serve; mkdocs build --strict и sphinx-build -W — предупреждение = ошибка, включена проверка якорей
7	Репозиторий	ExcaliBBur/static_site
8	GitHub Actions	pages.yml: build → deploy (upload-pages-artifact + deploy-pages), Pages → Source = GitHub Actions
9–10	Отечественные площадки	Helios (FreeBSD, nginx) и VPS (Ubuntu, Nginx Proxy Manager); helios.yml, vps.yml: rsync по SSH
11	Базовый URL	site_url/html_baseurl из переменной SITE_URL, своя для каждой площадки; use_directory_urls: false
12	Проверка	check_site.py после каждого деплоя: HTTP 200, контрольная строка, хеш коммита, индексы поиска, MathJax/Plotly с того же хоста
13	Лицензии	текст — CC BY 4.0, код — MIT, данные — CC0; CITATION.cff
14	Отладка	14 ошибок, основные — в разделе 6

Конвейер «данные → сайт». Скрипт prepare.py читает data/dataset.csv, обучает линейную регрессию градиентным спуском ($\eta = 0,01; 0,05; 0,2$) и МНК, строит графики и таблицу и пишет метку версии (коммит, дата, sha256 данных). build_all.py собирает MkDocs в корень и Sphinx в /sphinx/. Текст отчёта хранится один раз и копируется в оба сайта.

Базовый URL. Обычные страницы MkDocs ссылаются на ресурсы относительно и работают из любого подкаталога, но 404.html, sitemap.xml и rel=canonical строятся от site_url. Сборка с адресом GitHub Pages дала бы на Helios 404-страницу со ссылками на /static_site/assets/..., поэтому каждая площадка собирается со своим SITE_URL.

Способы публикации на GitHub Pages.

Критерий	Ветка gh-pages (peaceiris)	upload-pages-artifact + deploy-pages
Права токена	contents: write — весь репозиторий	pages: write, id-token: write
Компрометация шага	можно изменить код	можно только подменить сайт
Сторонний код	да	только официальные actions
История сайта	коммиты в ветке, репозиторий растёт	нет, репозиторий не растёт

Выбран второй способ. Все actions зафиксированы по SHA коммита.

Безопасность доставки на Helios и VPS. Для каждой площадки создан отдельный deploy-ключ ed25519, ограниченный на сервере: `restrict,command="rsync <каталог сайта>".` С ним возможен только rsync в каталог сайта; попытка выполнить команду даёт `rsync error: SSH_ORIGINAL_COMMAND does not run rsync.` Ключ хоста сверен с отпечатком из доверенного `known_hosts` и передаётся секретом, `StrictHostKeyChecking=yes`. На VPS каталог отдаётся через штатный файл `NPM data/nginx/custom/server_proxy.conf` (location `^~ /static_site/`, gzip, своя 404).

3. Пайплайны

Ключевой workflow `pages.yml` (сокращён; полные тексты `pages.yml`, `helios.yml`, `vps.yml`, `lighthouse.yml` — в репозитории):

```
on: {push: {branches: [main]}, pull_request: {}, workflow_dispatch: {}}
permissions: {contents: read} # минимум по умолчанию
concurrency: {group: pages, cancel-in-progress: false}
jobs:
  build:
    # на PR — только сборка
    steps:
      - uses: actions/checkout@3d3c42e... # v7.0.1, no SHA
      - uses: actions/setup-python@5fda3b9... # v7.0.0, cache: pip
        with: {python-version: "3.13.3", cache: pip}
      - run: pip install -r requirements.txt
      - run: python scripts/build_all.py # --strict / -W
      - run: python scripts/check_site.py http://127.0.0.1:8000/ --commit "${GITHUB_SHA::7}"
      - uses: actions/upload-pages-artifact@fc324d3... # v5.0.0
  deploy:
    if: github.event_name != 'pull_request' && github.ref == 'refs/heads/main'
    needs: build
    permissions: {pages: write, id-token: write}
    steps:
      - uses: actions/deploy-pages@368f825... # v5.0.1
      - run: python3 scripts/check_site.py "$SITE_URL" --commit "${GITHUB_SHA::7}" --wait-commit 180
```

Доставка в `helios.yml` и `vps.yml` отличается шагом:

```
test -n "$SSH_KEY" || { echo "::error::secret is empty"; exit 1; }
rsync -rlz --checksum --delete-after --delay-updates --chmod=D755,F644 \
-e "ssh -p 2222 -i ~/.ssh/helios -o StrictHostKeyChecking=yes" \
public/ "s335989@se.ifmo.ru:./" # "/" — каталог, заданный rsync
```

`--delay-updates` и `--delete-after` переименовывают новые файлы разом в конце передачи: при обрыве на сервере остаётся прежняя целая версия.

Результаты CI (один коммит, длительность job из API GitHub):

Площадка	Job целиком	Доставка	Healthcheck	Push → сайт
GitHub Pages	43 c (build) + 17 c (deploy)	7 c	2 c	≈ 70 c
Helios	62 c	3 c	18 c	≈ 65 c
VPS	70 c	5 c	22 c	≈ 75 c

Кэш `pip` почти не ускоряет установку (20 с без кэша, 16–19 с с кэшем): кэшируются скачанные пакеты, а время уходит на установку `numpy`, `matplotlib`, `plotly`. Первые запуски были проваленными (Pages не включён, секреты не заданы) — разбор в разделе 6.

4. T5. Публикуемость и цитируемость

Сайт становится публикацией, когда результат **идентифицируем** (DOI, ORCID), **неизменен для процитированной версии** (версии сайта, метка коммита и данных) и **машиночитаем** (мета-теги, `sitemap`, лицензии). Генератор отвечает только за последнее и частично за второе; DOI выдаёт Zenodo по GitHub Release (DOI должен вести на Zenodo, а не на сайт).

Шкала: **К** — из коробки, **П** — плагин, **Р** — вручную.

Механизм	MkDocs + Material	Sphinx + MyST	Pelican	Quarto
Стабильные URL, проверка ссылок	К	К	К	К
Редиректы со старых URL	П	П	Р	К
Версии сайта с переключателем	П (mike)	П (sphinx-multiversion)	Р	Р
Метка коммита и данных	Р	Р	Р	Р
DOI / ORCID / блок «Как цитировать»	Р	Р	Р	К
Highwire Press (Google Scholar)	Р	Р	Р	К
Dublin Core	Р	Р	Р	Р
Open Graph	К (плагин темы)	П	П	К
<code>sitemap.xml</code> / <code>robots.txt</code>	К / Р	П / Р	П / Р	К / Р
Раздельные лицензии	Р	Р	Р	Р

В проекте Highwire, Dublin Core и Open Graph добавлены переопределением одного блока шаблона (`overrides/main.html` в MkDocs, `_templates/page.html` и `sphinxext-opengraph` в Sphinx), метаданные для Zenodo — в `CITATION.cff`. Лицензии раздельные: CC-лицензии не рассчитаны на код, CC0 для данных снимает вопрос цепочки атрибуций. FAIR — свойство всей системы (репозиторий + архив + сайт), а не генератора.

Helios (проверено запросами к серверу, nginx 1.30):

Механизм	Helios	VPS
Статика, поиск, формулы без CDN, мета-теги	да	да
Сжатие <code>gzip</code>	нет (Plotly 4,7 МиБ)	да (1,7 МиБ)
Своя страница 404, редиректы 301	нет (<code>.htaccess</code> не работает)	да
<code>robots.txt</code>	нет (только в корне <code>se.ifmo.ru</code>)	да
Постоянный URL, свой домен	нет (адрес привязан к учётной записи)	пока оплачен сервер

Вывод: Helios — зеркало, не место канонической публикации.

5. P2. Стресс-тест научного контента

Одна страница на MkDocs + Material и Sphinx + MyST: нумерованные формулы со ссылками, таблица с объединёнными ячейками и подписью, график Matplotlib и Plotly, листинг с номерами строк, цитирование из `.bib` со списком литературы, две колонки, сноски, ссылка на другой раздел.

Замеры (Windows 11, Chrome через Playwright; сборка — среднее из 3):

Показатель	MkDocs + Material	Sphinx + MyST
Холодная сборка, с	1,02	2,42
Инкрементальная сборка, с	1,02	1,99
Вес страницы без сжатия, КиБ	7 530	7 177
— из них Plotly.js / MathJax, КиБ	4 707 / 2 059	4 707 / 2 059
Передано с gzip (GitHub Pages), КиБ	2 316	2 244
Внешних запросов	2 (api.github.com, от repo_url)	0
Формулы без CDN (все внешние запросы заблокированы)	все отрисованы	все отрисованы
Мобильные 390 px: горизонтальная прокрутка / колонки	нет / 1	нет / 1
Lighthouse mobile: Performance, A11y, BP, SEO	42, 87, 96, 100	51–71, 91, 96, 100
Lighthouse desktop: Performance	90	80–83

Вес страницы определяют библиотеки, а не генератор: Plotly.js — 63–66 %, MathJax — 27–29 %; тот же график в PNG весит 69 КиБ. Lighthouse запущен в CI (PageSpeed API без ключа исчерпал квоту); разброс между прогонами велик (Sphinx mobile: 71 и 51), нужна медиана нескольких прогонов.

Оформление. Стандартный вид тем заменён собственным без форка: палитра «подсолнух + малина» (тёмная тема — фиолетово-графитовая), шрифты Golos Text и Unbounded с кириллицей, лежат в самом сайте (139 КиБ). В MkDocs — `primary/accent: custom` и переопределение CSS-переменных Material, в Sphinx — `html_theme_options Furo`.

Что не удалось и ценой чего:

Требование	MkDocs + Material	Sphinx + MyST
Нумерация формул	нумерует MathJax в браузере; ссылки между страницами невозможны	из коробки
Объединённые ячейки	ручной HTML	rST grid table внутри MyST
Ссылка «Рисунок N»	номер пишется вручную	{numref}
Подпись к Plotly	есть	не нумеруется (figure только для изображений)
Библиография	плагин; смешивается со сносками	«автор, год», отдельно от сносок
Две колонки	6 строк CSS	sphinx-design, из коробки
Без внешних запросов	font: false + свои шрифты, локальный MathJax, остаётся GitHub API	локальные MathJax и шрифты

6. Отладка

1. Формулы выводились сырым TeX. Гипотеза: MathJax не загрузился — отвергнута (`tex-svg.js` → 200, `MathJax.version` = 3.2.2). В консоли `MathJax.config.tex.inlineMath = [["(", ")"]]`; дамп файла показал `"\("` вместо `"\\("` — heredoc оболочки съел слеш. Решение: файл переписан, в `measure.py` добавлена проверка «0 формул без отрисовки».

2. `mkdocs build --strict`: ...contains a link '../report/t5.md#1-постоянные-ссылки-...', but the doc does not contain an anchor. Гипотеза: стандартный slugify вырезает кириллицу. Проверка: у заголовка `id="1-url"`. Решение: `pydownx.slugs.slugify(case=lower)` — id совпали с MyST.

3. Plotly шириной 480 из 882 px. Гипотеза «виноват <figure>» после замены на <div> не подтвердилась. Замер в DOM: блок `/// figure-caption` сам оборачивает `iframe` в

<figure> шириной по содержимому. Решение: `.md-typeset figure:has(.plot-frame) { width: 100% }`.

4. Проваленные запуски CI. `deploy-pages: Failed to create deployment (status: 404) ... Ensure GitHub Pages has been enabled; API: "has_pages": false` → включён Pages. `helios: exit code 255 от ssh при пустых секретах` → секреты добавлены, в workflow — `test -n "$SSH_KEY"`.

5. Lighthouse: FCP 9,5 с у MkDocs (Sphinx — 1,0 с). Гипотеза: MathJax 2 МиБ подключён синхронно. Проверка HTML: у MkDocs `<script src=...tex-svg.js>`, у Sphinx `defer`. Решение: `defer: true` в `extra_javascript` — FCP 0,9 с, но TBT вырос: выполнение MathJax переместилось после отрисовки.

6. Лимиты API. `PageSpeed: 429 Quota exceeded ... Queries per day; GitHub API: 403 rate limit exceeded (X-RateLimit-Remaining: 0)` от цикла ожидания CI. Решение: Lighthouse в CI, ожидание — по веб-странице Actions.

7. Вывод

Рекомендуемый стек для публикации результатов вычислительных экспериментов: **Sphinx + MyST** (Furo, sphinxcontrib-bibtex, sphinx-design) — нумерация и ссылки на формулы, рисунки, таблицы при сборке, библиография «автор, год», ноль внешних запросов; скрипт «данные → графики → страница» перед сборкой; локальный MathJax; GitHub Actions с `deploy-pages` и `healthcheck`; GitHub Pages — основной хостинг, Helios/VPS — зеркала; DOI через Zenodo, CITATION.cff, отдельные лицензии. Графики — PNG, Plotly только там, где интерактивность нужна для анализа.

Условие	Рекомендация меняется на
Результаты — Jupyter-ноутбуки, исполняемые при сборке	Sphinx + MyST-NB / Jupyter Book
Нужна «публикация из коробки» (цитирование, ORCID, Highwire)	Quarto
Документация кода, важны поиск и интерфейс, формул мало	MkDocs + Material (+ mike)
Нужна PDF-версия	Sphinx (LaTeX) или Quarto
Только отечественная инфраструктура	та же сборка в GitVerse/SourceCraft, хостинг с доменом
Результат должен цитироваться годами	обязательно Zenodo и собственный домен